

Same Request, Different Answer: Quantization Amplifies Cache-Induced Divergence in LLM Serving

ADITI PATODIYA

Independent Researcher, Milpitas, CA 95035 USA

Corresponding author: Aditi Patodiya (e-mail: aditi.patodia31@gmail.com). This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible.

ABSTRACT Prefix caching, in which a serving engine reuses the key and value tensors of a shared prompt prefix across requests, is enabled by default in the major open-source stacks and treated as a transparent optimization. We measure what it costs in reproducibility, and find that the cost rises sharply with weight quantization. Holding the model, decoding parameters, seed, and request order fixed, and issuing every request serially at batch size one, we ran an eighty-episode multi-turn agentic tool-use workload with caching enabled and disabled across two engines and four weight formats. Enabling the cache changed the agent's trajectory on 36.2 percent of episodes at 16-bit precision and on 75.0 percent at four-bit, a gradient that survives re-measurement under a controlled cache configuration. With caching disabled, repeated execution was bit-identical in every configuration, 0 of 800 episodes, which bounds other sources of nondeterminism at 0.5 percent. Repeated cache-enabled runs did diverge, and three experiments locate the cause: a single server-level prompt-cache setting moves run-to-run divergence by 37.5 percentage points, execution order acts only while that setting is active, and restoring cache state makes the cached and recompute paths each reproduce on 40 of 40 items while still differing from each other on 14. Cached serving is deterministic given cache state, and irreproducible in practice because that state is absent from the request and never reset by default. A single-turn bridge shows the divergence reaching task outcomes without shifting aggregate accuracy. We release the harness, logs, and analysis pipeline.

INDEX TERMS Large language models, inference serving, reproducibility, prompt caching, key-value cache, quantization, software testing, empirical software engineering

I. INTRODUCTION

Prefix caching is one of the least controversial optimizations in modern language model serving. When consecutive requests share a leading span of tokens, the server reuses the key and value tensors it already computed for that span instead of recomputing them. Every major serving stack ships the feature and most enable it by default [1]–[3]. The savings are large and well documented: a recent evaluation of long-horizon agent pipelines reports 41 to 80 percent lower cost and 13 to 31 percent faster time to first token when caching is on [4]. For agentic workloads the appeal is even stronger than for chat, because an agent re-sends its entire growing transcript on every step, so almost the whole prompt is

a cache hit after the first turn.

The optimization carries an implicit promise. Reusing arithmetic that was already performed is supposed to be an implementation detail, invisible above the serving layer. A user who sends the same request with the same sampling parameters expects the same answer whether or not the server happened to have a warm cache.

That promise does not hold, and the failure is not subtle. Reusing cached keys and values changes the order in which floating point accumulations are performed, and floating point addition is not associative, so the reused path and the recomputed path produce slightly different numbers. Most of the time the difference is invisible. Occasionally it moves a token probability across

a decision boundary, and the sampled token changes. In a single-turn setting that is a curiosity. In an agent loop it is not, because the changed token may be part of a tool call, and the tool call determines the next observation, which determines the rest of the episode.

Practitioners have noticed. Two issues in the vLLM tracker report behavior of this kind. One describes an accuracy drop from roughly 78 percent to 60 percent when the prefix cache interacts with the recomputation path [5]. The other, filed against ROCm hardware and not reproducing on CUDA, shows an identical request returning different text depending on whether it hit the cache [6]. Both were closed without a systematic characterization. In the literature, one recent study established that cached and recomputed decoding are not numerically equivalent under FP16 on single-turn arithmetic [7], and a study of inference backends found that backend choice alone can shift benchmark scores by as much as 16.6 points, naming prefix caching among the causes without isolating it [8]. What has been missing is a controlled measurement of how often this matters for the workload the feature was designed to accelerate, and of what the effect does when the weights are quantized, which is how most locally served models run.

This paper provides that measurement. We run a paired design in which the only variable is the cache setting, with greedy decoding, a fixed seed, a batch size of one, and serial requests, so the usual sources of serving nondeterminism are held out. The workload is multi-turn agentic tool use from the Berkeley Function Calling Leaderboard [9], with single-turn grade school mathematics [10] as a bridge to the prior single-turn result. We run every configuration twice, which turns the design into something stronger than a comparison between arms: it lets each arm be compared against itself.

That within-arm comparison produces the paper's central result. With caching disabled, repeated execution of the identical workload is bit identical, in every configuration we tested, without exception. With caching enabled, the same repetition changes the outcome of a large fraction of episodes. The system is not noisy in general. It becomes noisy exactly when the cache is turned on. The correct description of the effect is therefore not that cached and uncached execution differ, but that caching makes the server's output a function of its own recent history rather than of the request alone.

The mechanism is not news, and we do not present it as such. Work on deterministic inference already identifies reduction-order variation between prefilled and cached execution as a source of nondeterminism, and prescribes batch-invariant kernels to remove it [11]. SGLang now ships a deterministic mode that reports consistent output across cached and uncached prefill on two of its three attention backends [12]. What has not been established is the magnitude in the configuration

people actually deploy. Those fixes are opt-in, they carry a throughput cost, they are unavailable on some backends and absent entirely from llama.cpp, and they are off by default in every stack we tested. Our contribution is to measure what the default configuration does to the workload the cache was built for.

The contributions are as follows.

- A controlled measurement across model families, quantization formats, and two serving implementations, showing that cache-disabled serving is bit-identical across repeated runs while cache-enabled serving is not.
- An isolation of the cause. Execution order is ruled out, a single server-level prompt-cache setting is shown to move run-to-run divergence by 37.5 percentage points, and a reset control demonstrates that the cached path is reproducible once cache state is restored. The resulting claim is that cached serving is deterministic given cache state, which deployments neither report nor control.
- The first characterization of how weight quantization interacts with cache-induced divergence, showing that coarser quantization amplifies it substantially.
- An outcome-level analysis separating instability from degradation, independently replicating a concurrent single-turn result in a serving-engine setting: individual items change correctness in both directions while aggregate accuracy does not move.
- A released harness, raw per-request logs including server-reported cache exposure, and an analysis pipeline that regenerates every number in this paper from those logs.

II. BACKGROUND AND RELATED WORK

A. PREFIX CACHING IN PRODUCTION SERVING STACKS

Key-value caching within a single generation is standard. What concerns us here is reuse across requests. PageAttention introduced the memory management that makes such sharing practical by storing the cache in non-contiguous blocks that different sequences can reference [1]. RadixAttention generalized the idea to a prefix tree over live requests, which is a natural fit for agent loops where many requests share long leading spans [2]. Prompt Cache went further and reused attention states for non-contiguous prompt modules [3], and CacheBlend fused cached knowledge for retrieval-augmented serving with selective recomputation [13]. Cache eviction and windowing schemes add another layer of state that varies between runs [14], [15].

These systems are evaluated on throughput, latency, and memory, and often on aggregate task accuracy. What they do not report is whether a given request returns the same answer with a warm cache as with a cold one. The closest evaluation to ours in workload terms

measured caching across more than five hundred agent sessions but recorded only cost and time to first token [4]. The behavioral question was left open, and the field guidance that resulted, namely to cache aggressively, was issued without it.

B. NONDETERMINISM IN LLM INFERENCE

Practitioners have long observed that temperature zero does not guarantee identical output, and this has been documented systematically [16]. The mechanism that has received the most recent attention is batch invariance. Because reduction order in fused kernels depends on how requests are grouped, a request batched with different neighbors takes a different arithmetic path, and rewriting kernels to be batch invariant removes that source [11]. SGLang implemented such a deterministic mode, and it is instructive that the accompanying benchmarks were run with the radix cache disabled [12]. Determinism and caching have not yet been reconciled in production stacks. A complementary line restores determinism through verified speculation rather than kernel redesign [17].

Our measurements are deliberately positioned outside the batch invariance story. Every request in this study is issued alone, at batch size one, in serial order, so grouping cannot vary between runs. Any divergence we observe therefore has a different origin, and the cache-disabled arms confirm this directly by reproducing bit-identically under the same conditions.

Two studies bear directly on the cached path. One demonstrated that FP16 decoding with a key-value cache is not numerically equivalent to recomputation, using single-turn arithmetic and three models, and attributed the effect to accumulation order under FP16 non-associativity [7]. Another quantified how much benchmark scores move when only the inference backend changes, reporting shifts up to 16.6 points and naming prefix caching among the mechanisms [8]. We extend the first from single-turn text to multi-turn agent episodes and add quantization as a factor, and we isolate the variable that the second identified but did not separate.

C. QUANTIZATION AND BEHAVIORAL CHANGE

Quantization studies usually report aggregate accuracy, and by that measure modern low-bit formats are close to lossless [18]. A more careful reading of compressed model behavior shows that this framing hides churn: models that match a baseline on aggregate accuracy can disagree with it on a substantial share of individual items [19]. Damage from quantization is also unevenly distributed across populations that aggregate metrics average over [20].

That distinction is the one our results turn on. We do not claim that quantization degrades accuracy, and our data would not support such a claim. We report instead that quantization changes how easily a small numerical

perturbation from the cache path flips a decision, which is a property of the model's margin structure rather than of its average competence.

III. STUDY DESIGN

A. RESEARCH QUESTIONS

RQ1 (baseline determinism). With prefix caching disabled, is repeated execution of an identical workload bit-identical under greedy decoding on a fixed serving stack?

RQ2 (cache-induced divergence). With prefix caching enabled, how often does repeated execution of that same workload produce different outputs, and how deep into an episode does the first difference appear?

RQ3 (quantization interaction). Does coarser weight quantization amplify or damp the divergence measured in RQ2?

RQ4 (backend generality). Do the effects reproduce across independent serving implementations?

RQ5 (task outcomes). Does cache-induced divergence change task correctness, and if so, is the change directional or merely unstable?

B. EXPERIMENTAL PROTOCOL

The study uses a paired design. A cell is one complete pass of a workload through one serving configuration with prefix caching either enabled or disabled. Everything outside the caching setting is held fixed within a configuration: the model weights, the quantization format, the engine build, the sampling parameters, the request order, and the hardware.

Sampling is greedy throughout. Temperature is zero, the random seed is fixed at 42, and every request carries the same seed. Requests are issued serially with a batch size of one, so no request shares a forward pass with another. This matters because continuous batching is itself a documented source of run-to-run variation, and leaving it active would confound the measurement we are trying to isolate. The key-value cache is stored in 16-bit floating point in every arm, including the arms that serve quantized weights, so that cache precision never varies with the weight format.

Two properties of the design carry the argument. First, each configuration is run twice in full, which yields a within-arm comparison: the same arm against itself. The cache-disabled within-arm comparison is the internal validity check, since any difference there would indicate a nondeterminism source we failed to control. Second, the cache setting is verified from the server rather than assumed wherever the engine permits it. On llama.cpp every response records how many prompt tokens were served from cache, making cache exposure a measured per-request variable rather than an inferred property of the arm. vLLM 0.11.0 leaves that field unpopulated on the endpoint we use, so there the check

comes from the engine log instead, as Section IV-E reports.

Cache control differs by engine and we follow each engine's own mechanism. We use llama.cpp [21] with GGUF weights [22] and vLLM. In llama.cpp the cache is a per-request flag, so both arms run against a single server process. In vLLM prefix caching is a server-level setting, so each arm runs against its own server process launched with the corresponding flag. A third engine, SGLang, entered the frozen study plan as a budget-contingent stretch lane, which the plan specified would be the first component dropped if the compute allowance required it. It was dropped under that rule before any collection, so no SGLang cells exist and none are reported. Engine versions, launch flags, and model checksums are recorded for every run and released with the artifact.

All measurements ran on a rented NVIDIA RTX 4090 with 24 GB of memory, compute capability 8.9, under Ubuntu 24.04 with the CUDA 12.6 toolkit. The serving stacks are llama.cpp built from source at release b10434, commit 7e4c0a9, with CUDA offload, and vLLM 0.11.0 on PyTorch 2.8.0+cu128 with transformers 4.57. The grid of Table 2 ran under NVIDIA driver 570.211.01, and the experiments of Section IV-C ran under 580.126.20 on a second machine of the same type, so comparisons across those two groups carry a driver difference. Every comparison from which we draw a causal conclusion, including the flag and ordering experiments, was collected within a single machine and driver version.

C. WORKLOADS

The primary workload is multi-turn agentic tool use. We use the multi-turn base category of the Berkeley Function Calling Leaderboard, which places a model in a simulated environment of stateful APIs and scores whether the resulting environment state and call sequence match a reference. Episodes run for several user turns, and within each turn the agent may take many steps, so a single episode issues about ten requests on average, ranging from seven to thirteen across configurations, and accumulates a context in the thousands of tokens. That growth pattern is what makes the workload appropriate here: each step re-sends the whole conversation, which is exactly the shape of request that prefix caching is designed to accelerate. Episodes are selected by a rule fixed before any result was inspected, namely the first eighty entries in ascending numeric identifier order.

The secondary workload is single-turn grade-school mathematics from GSM8K. It serves two purposes. It provides a bridge to prior work on cached inference, which studied single-turn arithmetic, and it supplies an outcome measure clear of the floor: the models we run solve roughly nine in ten of these problems, which still leaves enough incorrect items for flips to be observable in both directions. Each item is presented four times in

a fixed order: twice on the recompute path and twice on the cache-hit path. The repeated passes on each path give a per-path determinism check, so a difference between paths is only counted as cache-attributable when each path agreed with itself.

D. ISOLATING THE CAUSE

Our first grid measured divergence but did not establish what produced it, and a measurement of that kind invites two objections: that the passes differed in execution order, and that comparing two runs whose cache states differ is definitionally guaranteed to show a difference. Three further experiments address both, all on Qwen2.5-7B at Q4_K_M under llama.cpp, the configuration with the highest measured divergence.

The first varies execution order alone. One arm places a complete cache-disabled pass between the two cache-enabled passes, matching the order used in our llama.cpp lane; the other runs the two cache-enabled passes adjacently, matching the vLLM lane. Everything else is held constant, including a fresh server for each arm.

The second varies one flag. llama.cpp maintains a host-memory prompt cache that is separate from the prefix cache under study, stores whole conversation states, and selects among them by prefix similarity rather than by identity. We run the same procedure with that layer disabled and at its default, again with a fresh server for each arm, so the flag is the only difference.

The third restores cache state directly. Each item is served four times, twice on the recompute path and twice on the cache-hit path, with the cold state re-established before each recompute pass. Comparing each path against itself under a restored state separates dependence on cache state from residual randomness in either path. We do not use the engine's cache-erase endpoint for this: it returns success while leaving the prompt cache intact, which we verified from the reported cached-token counts, so the cold state is established with the per-request flag that forces recomputation, whose effect is visible in the same telemetry.

E. MEASURES

Divergence is measured on token identifiers rather than on decoded text wherever the engine returns them, so that a difference is registered even when detokenization hides it. llama.cpp returns token identifiers on every response. vLLM 0.11.0 returns token strings on the endpoint we use, so its two configurations are compared on decoded tokens, which is a slightly weaker criterion; each configuration records which criterion applied. For a pair of runs we walk the request sequence of each episode in lockstep and record the first position where the two runs differ, either in the prompt that was assembled or in the emitted token sequence. An episode is counted as divergent if any request differs or if the two runs issue

different numbers of requests, the latter meaning the agent took a different number of steps.

Task outcomes are scored by the benchmark's own checker without modification, so scoring is independent of our instrumentation. For the agentic workload the checker compares the final environment state and the sequence of invoked methods against the reference. For the mathematics workload an answer is correct when the final numeric value matches the reference.

Proportions are reported with Wilson intervals, with one caveat we quantify rather than assume away. Episodes within a cell run in fixed order against one server, and this study's own claim is that state carries across requests, so episodes are not exchangeable and Wilson intervals may be anticonservative. We therefore also compute a moving-block bootstrap, which preserves dependence between neighbouring episodes, for the configuration that carries the mechanism result. It gives a wider interval, 20.0 to 55.0 percent against Wilson's 28.8 to 49.7, and we report the wider one where the distinction matters. We also test directly for position dependence, and report it below. Paired outcome changes are tested with an exact McNemar test [23] on the discordant pairs, which is the appropriate test when the same item is observed under both conditions and we care whether changes favor one condition [24]. We report the discordant counts alongside the p-value, since with small numbers of flips the counts themselves are more informative than the test statistic.

IV. RESULTS

A. RQ1: CACHE-DISABLED EXECUTION IS BIT-IDENTICAL

We begin with the control, because everything else depends on it. For every configuration we executed the full eighty-episode workload twice with prefix caching disabled and compared the two runs request by request on token identifiers.

Not one episode differed. In every configuration, across both serving engines, across weight formats from 16-bit floating point down to three-bit k-quantization, the second run reproduced the first exactly: identical prompts at every step, identical emitted tokens, identical numbers of requests per episode. Table 2 reports 0.0 in the cache-disabled column for every configuration. With zero events in 80 trials, the per-episode divergence rate is bounded at 4.6 percent with 95 percent confidence, and pooling the ten configurations gives 0 of 800 episodes, an upper bound of 0.5 percent. We state the bound rather than the point estimate throughout.

The value of this result is what it licenses. Under our conditions, greedy decoding with a fixed seed, a batch size of one, and serial requests, the serving stack is a deterministic function of its input to within the bound above. Any divergence observed elsewhere in this paper therefore cannot be attributed to sampling, to scheduler

nondeterminism, to batch composition, or to unspecified engine noise, because all of those are equally present in the cache-disabled arms and produced no observed variation. We also checked every episode log for failures: no episode in any cell raised an error, so divergence cannot be an artifact of timeouts or exceptions.

B. RQ2: CACHE-ENABLED EXECUTION DIVERGES, AND WHY

We then repeated the identical procedure with prefix caching enabled, changing nothing else about the workload or the decoding parameters.

Two distinct measurements follow, and separating them is the substance of this section. The first compares the cache-enabled arm against the cache-disabled arm. The second compares the cache-enabled arm against itself, re-run.

1) Cache-enabled and cache-disabled execution differ

The cross-arm column of Table 2 reports the first comparison. Enabling the cache changes the agent's trajectory on between 36.2 and 91.2 percent of episodes, depending on configuration. This measurement is stable: when we later re-ran configurations under a deliberately controlled cache setup (Section IV-C), the cross-arm rates reproduced within a few points, 81.2 against 75.0 percent at Q4_K_M and 77.5 against 77.5 percent at Q3_K_M.

The mechanism is the familiar one. A request that hits the cache reads keys and values for the shared prefix instead of recomputing them, the accumulation order in the attention computation changes, floating point addition is not associative, and the resulting logits differ in their low-order bits. What the measurement adds is that the difference is not confined to low-order bits of the output. It crosses token decision boundaries often enough to change agent behavior in a large fraction of episodes. Divergence also begins early: across configurations the first differing request has a median index between 1 and 4 of roughly ten, and within that request the first differing token has a median index between 3.5 and 17. An episode rarely survives its opening exchanges unchanged, which is why per-request divergence rates that look small compound into episode rates that do not.

2) A second caching layer, not the prefix cache, drives run-to-run divergence

Re-running the cache-enabled arm against itself changed the trajectory on 8.8 to 77.5 percent of episodes across the ten configurations of Table 2. Taken alone, that invites the conclusion that cached serving is intrinsically nondeterministic.

It is not, and the next section shows why. Those ten configurations were collected with llama.cpp's server-level prompt cache left at its default, a second caching

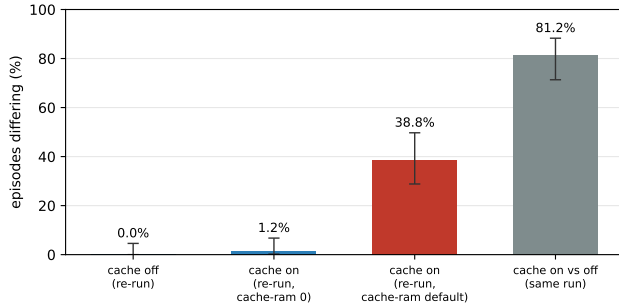


FIGURE 1. Where the divergence comes from. Repeating a cache-disabled run changes nothing. Repeating a cache-enabled run changes little once the server-level prompt cache is disabled, and a great deal at its default setting. Comparing the cache-enabled and cache-disabled arms of the same run differs from all three. All four bars come from the controlled runs of Section IV-C rather than from Table 2. Bars show 95% Wilson intervals; Qwen2.5-7B at Q4_K_M under llama.cpp, 80 episodes per bar.

layer, a host-memory prompt cache that is distinct from the prefix cache under study and that persists whole conversation states across requests independently of it. Once that layer is disabled, repeated cache-enabled runs of the same configuration agree on 79 of 80 episodes. The cached path is reproducible; what is not reproducible is the state it reads from.

C. THE MECHANISM: STATE CARRIED BETWEEN RUNS

Three experiments isolate the cause. All use Qwen2.5-7B at Q4_K_M, the configuration with the highest measured divergence, and all hold the model, episode set, decoding parameters, and server freshness constant. The vLLM comparison at the end of the section uses the same model at FP16, since vLLM serves the unquantized weights.

Figure 1 summarizes the result.

A second caching layer controls the effect. Beyond the per-request prefix cache under study, llama.cpp maintains a host-memory prompt cache that stores whole conversation states, selects among them by longest-common-prefix similarity rather than by identity, and evicts oldest-first [25]. It is enabled by default at 8192 MiB and controlled by a documented flag [26], and it was introduced in October 2025, so evaluations run before that date were unaffected and those run after have been affected silently. Our first grid left it at its default. Holding everything else fixed, including a fresh server for each arm, and changing only that flag, repeated cache-enabled runs diverged on 1 of 80 episodes with the layer disabled (1.2 percent, 95 percent CI 0.2 to 6.8) and 31 of 80 with it at its default (38.8 percent, CI 28.8 to 49.7). One documented configuration setting accounts for a difference of 37.5 percentage points in run-to-run reproducibility. We state the difference rather than the ratio, since a single divergent episode in the disabled arm cannot pin a ratio down.

The setting and its default are documented, and the

documentation separately warns that the per-request prefix cache can produce nondeterministic results [26]. What we did not find in the documentation, in the pull request that introduced the feature, or in the issue tracker is any report that this second layer is an independent source of run-to-run variation on an identical workload. Our contribution here is the measurement rather than the discovery of the setting: a request's arithmetic path depends on which stored state the similarity search selects, and that depends on what the server processed earlier.

Execution order acts through the same layer. Our llama.cpp and vLLM lanes had run their passes in different orders, so we tested order against the flag above. Table 1 assembles the four cells. Three are new runs made for this purpose; the fourth, at the default setting with a cache-disabled pass in between, is the corresponding configuration from Table 2, whose first cache-enabled pass we verified is identical to that of the new run in the same row. We label it as such rather than presenting the table as a single designed experiment.

With the prompt-cache layer active, interposing a complete cache-disabled pass between the two cache-enabled passes raises divergence from 38.8 to 77.5 percent, consistent with the intervening traffic rewriting the state that the second pass inherits. With the layer disabled the same manipulation changes nothing, 1.2 percent either way, and the two orderings produced byte-identical output in every cell. Order therefore appears to act through the state the layer carries rather than independently of it. The two rows do not share a common starting point: at the default setting the first pass issues 854 requests and with the layer disabled it issues 827, so the setting changes behavior within a pass as well as between runs.

Divergence in that configuration is also concentrated early in the run. Split into quartiles of execution order, the counts of divergent episodes are 16, 6, 6 and 3, a trend that a permutation test puts at $p = 0.0002$. The same split in the grid configuration shows no trend ($p = 0.59$). A layer that accumulates and evicts entries as a run proceeds would be expected to produce exactly this settling behaviour, and it is a further reason to treat episodes within a cell as dependent rather than as independent trials.

With cache state restored, the cached path is reproducible. The reset control serves each of 40 items four times, twice on the recompute path and twice on the cache-hit path, re-establishing the cold state before each recompute pass and verifying it from the reported cached-token counts, which are zero on every cold request and positive on every warm one. The recompute path reproduced on 40 of 40 items and the cached path on 40 of 40, while the two paths differed from each other on 14 of 40. This control was run with the prompt-cache layer disabled, so it establishes that the cached path is

TABLE 1. Execution order and the server-level prompt cache, crossed. Values are the fraction of 80 episodes whose trajectory changed when the cache-enabled arm was repeated. Order matters only while the prompt-cache layer is active. Qwen2.5-7B at Q4_K_M under llama.cpp.

	passes adjacent	cache-off pass in between
prompt cache at default	38.8%	77.5%
prompt cache disabled	1.2%	1.2%

a function of cache state in the regime where state is otherwise controlled; it does not by itself speak to the default configuration, which the first two experiments cover.

The second engine points the same way through a different lever. vLLM exposes no equivalent of the llama.cpp setting, so what varies there is server lifetime. Configurations that launched a fresh server for each cache-enabled pass diverged on 0 of 80 episodes, since each pass then begins from an empty cache. Configurations that served both passes from one process diverged on 7 of 80 and 22 of 80 for Qwen2.5-7B in two sessions, and on 8 of 80 for Llama-3.1-8B in its one session. The direction is consistent across every session; the magnitude for the repeated model is not, 8.8 against 27.5 percent. Those two sessions differ in the script that launched them, in whether an explicit GPU memory fraction was set, and in the machine and driver version, and we did not isolate which of these matters. We therefore take the vLLM lane as establishing the direction of the effect on a second engine and the llama.cpp lane as fixing its size.

Taken together, these results support a more precise claim than the raw within-arm numbers suggest. Cached serving is a deterministic function of the request and the cache state. It is irreproducible in practice because cache state is not part of the request, is not reported in the response, and is not reset between runs by default in any stack we tested. Disabling the cache removes the state, and with it the divergence. That gap, between a system that is deterministic and one that is reproducible, is what these experiments measure.

D. RQ3: QUANTIZATION AMPLIFIES DIVERGENCE

Figure 2 plots the gradient. We state it on cross-arm divergence, the measurement Section IV-C showed to be stable under cache configuration. Reading Table 2 down the weight-format column for Qwen2.5-7B under llama.cpp, cross-arm divergence rises from 36.2 percent at 16-bit floating point to 61.3 percent at Q8_0, 75.0 percent at Q4_K_M and 77.5 percent at Q3_K_M. Re-measuring three of those cells with the server-level prompt cache disabled reproduced the pattern closely: 40.0 percent at 16-bit, 81.2 percent at Q4_K_M and 77.5 percent at Q3_K_M. The gradient is a property of the cache-versus-recompute contrast, not of the configuration artifact identified above. Llama-3.1-8B rises from

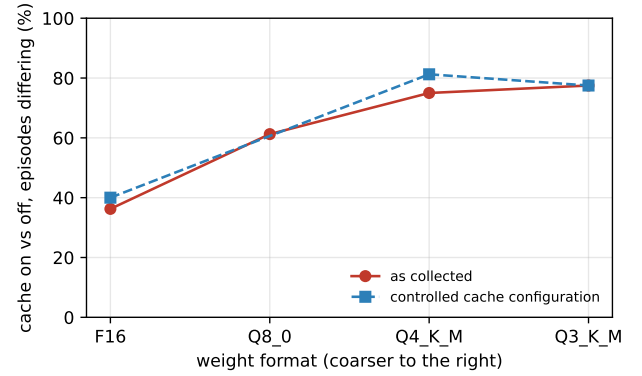


FIGURE 2. Coarser weight quantization widens the gap between cache-enabled and cache-disabled execution. The pattern holds when three of the cells are re-measured with the server-level prompt cache disabled, so it is a property of the cache-versus-recompute contrast rather than of the configuration artifact discussed in Section IV-C. Qwen2.5-7B under llama.cpp, 80 episodes per point.

55.0 percent at Q8_0 to 70.0 percent at Q4_K_M, with Q3_K_M at 68.8 percent, within a few points of the cell above it. The 16-bit cell matters most for interpretation: divergence is already present without any quantization, so quantization is not the cause of the effect. It is a multiplier.

The trend across ordered formats is significant: a Cochran–Armitage test on the four Qwen2.5-7B cells gives $z = 5.68$, $p = 1.3 \times 10^{-8}$.

Amplification saturates at the coarsest settings. Q4_K_M and Q3_K_M sit within a few points of each other in both the original and the controlled measurements and their intervals overlap, so the ordering of those two cells carries no weight. Below four bits the outputs are already constrained enough by quantization error that additional perturbation from the cache path has less room to change the argmax.

The amplification is consistent with margin structure. Coarser quantization compresses the gaps between competing token logits, so when candidates sit closer together a smaller numerical perturbation is enough to reorder them and the same cache-induced difference flips more decisions. This is consistent with the observation that compressed models can match a baseline on aggregate accuracy while disagreeing with it on many individual items [19].

E. RQ4: THE EFFECT IS NOT IMPLEMENTATION-SPECIFIC

Two independently built engines agree on both halves of the result. The last two rows of Table 2 carry it. Under vLLM, an engine with a different cache design, different kernels, and chunked prefill enabled, the cache-disabled arms are again bit-identical across repeated runs, and the cache-enabled arms are again not.

The magnitudes differ substantially and we report that rather than smoothing it. On vLLM at 16-bit,

TABLE 2. Episode-level divergence. Each configuration ran the same 80-episode workload twice per arm. *Cache off* and *cache on* report the fraction of episodes whose trajectory changed when the same arm was repeated; *cross-arm* compares the two arms. Every cache-off value is exactly zero, which bounds all other sources of nondeterminism under these conditions.

Engine	Model	Weights	n	re-run divergence (%)			cross-arm (%)
				cache off	cache on	95% CI	
llama.cpp	Qwen2.5-7B	F16	80	0.0	20.0	[12.7, 30.0]	36.2
llama.cpp	Qwen2.5-7B	Q8_0	80	0.0	55.0	[44.1, 65.4]	61.3
llama.cpp	Qwen2.5-7B	Q4_K_M	80	0.0	77.5	[67.2, 85.3]	75.0
llama.cpp	Qwen2.5-7B	Q3_K_M	80	0.0	67.5	[56.6, 76.8]	77.5
llama.cpp	Llama-3.1-8B	Q8_0	80	0.0	33.8	[24.3, 44.6]	55.0
llama.cpp	Llama-3.1-8B	Q4_K_M	80	0.0	60.0	[49.0, 70.0]	70.0
llama.cpp	Llama-3.1-8B	Q3_K_M	80	0.0	62.5	[51.5, 72.3]	68.8
llama.cpp	Qwen2.5-14B	Q4_K_M	80	0.0	31.2	[22.1, 42.1]	91.2
vLLM	Qwen2.5-7B	FP16	80	0.0	8.8	[4.3, 17.0]	62.5
vLLM	Llama-3.1-8B	FP16	80	0.0	10.0	[5.1, 18.5]	53.8

TABLE 3. Single-turn bridge (GSM8K). Each item is served four times, twice on the recompute path and twice on the cache-hit path. Both paths are individually deterministic, yet they disagree with each other on a large fraction of items. Correctness flips occur in both directions. Answers are derived from the stored responses by numeric comparison; the released artifact reproduces every value.

Model	Weights	n	path determinism		cross-path divergence		flips (n)	correct		McNemar p
			cold	warm	%	95% CI		cold	warm	
Qwen2.5-7B	F16	200	200/200	200/200	3.5	[1.7, 7.0]	0	183	183	1.000
Qwen2.5-7B	Q8_0	200	200/200	200/200	44.5	[37.8, 51.4]	4	184	182	0.625
Qwen2.5-7B	Q4_K_M	200	200/200	200/200	41.5	[34.9, 48.4]	4	180	180	1.000
Qwen2.5-14B	Q4_K_M	200	200/200	200/200	48.0	[41.2, 54.9]	1	185	186	1.000
Qwen2.5-7B	Q4_K_M	500	500/500	500/500	45.4	[41.1, 49.8]	12	453	459	0.146
Qwen2.5-7B	Q3_K_M	200	200/200	200/200	48.0	[41.2, 54.9]	3	178	179	1.000

repeating the cache-enabled arm changed 8.8 percent of episodes, against 20.0 percent for llama.cpp at the same precision, yet the cross-arm comparison on vLLM is high at 62.5 percent. The combination says that vLLM’s cached path is comparatively self-consistent while still differing from its own recompute baseline on most episodes. Implementations differ in block reuse policy, in kernel selection, and in whether prefill is chunked, and any of these could plausibly account for the gap. We measured the rates; we did not isolate the cause, and we do not speculate further.

One disclosure belongs with these vLLM figures. The cache-enabled and cache-disabled arms of the Qwen2.5-7B configuration were launched by different scripts, and the relaunch set an explicit GPU memory fraction that the original did not. That parameter governs the block pool the cache draws on, so we checked whether it mattered: the cache-disabled arm reproduces byte-identically across all three sessions, including the differently provisioned ones, so the comparison is unaffected. The manipulation check itself comes from the engine rather than from the response body. vLLM 0.11.0 does not populate the cached-token field on the completions endpoint, so every response in the cache-enabled arm reported zero cached tokens even while the cache was working. The engine log resolves it: prefix caching is recorded as enabled at startup and the reported hit rate rises from 87.1 to 99.1 percent across 226 logged observations. We note this because a study that trusted

response telemetry alone would have concluded that the manipulation had failed and discarded a valid arm.

F. RQ5: INSTABILITY WITHOUT DIRECTIONAL BIAS

The agentic workload cannot answer the outcome question. Models of this size solve few of its episodes, from 1.2 percent in the weakest configuration to 18.8 percent for Qwen2.5-14B, so success rates sit near the floor and paired flips are too few to support a statistical test: pooled across all ten configurations the cache-enabled arm won 12 episodes and the cache-disabled arm won 7, and no per-configuration test approaches significance. The largest model we ran raises the ceiling without changing the conclusion, which suggests the limit is task difficulty rather than a quirk of the smallest models. We therefore draw no conclusion about agent task success from these data and report the trajectory-level results of Sections IV-A–IV-E, which do not depend on task success, as the agentic contribution.

Outcome conclusions come from the single-turn bridge, where accuracy sits well clear of the floor at 89 to 93 percent. Table 3 reports it. Each item is served four times, twice on the recompute path and twice on the cache-hit path, so that a cross-path difference is only counted when each path first agreed with itself. Every path was internally deterministic on every item in every configuration, which is what licenses the attribution.

The 500-item run at Q4_K_M extends the 200-item run at the same setting, sharing its first 200 items, so

we pool over distinct items rather than over table rows: 1300 items across five distinct configurations, with 20 correctness flips attributable to the cache path.

Two patterns appear. The first is a dose-response that mirrors Section IV-D. At 16-bit precision the cache path changes the output for 3.5 percent of items, so at full precision a single-turn request is largely unaffected. Under quantization the same measurement rises to between 41.5 and 48.0 percent, and a larger run of 500 items at Q4_K_M places the rate at 45.4 percent with a correspondingly tighter interval. Comparing this against the agentic numbers is instructive: at 16-bit, individual requests diverge on 3.5 percent of items, yet 36.2 percent of agent episodes do under the same comparison, because an episode chains about ten requests and a single changed token propagates through the remaining turns. Quantization and episode length act as independent amplifiers of the same underlying perturbation.

The second pattern is the more important one for practice. Correctness flips occur in both directions and aggregate accuracy does not move. Of the 20 flips, 13 favor the cached path and 7 favor recompute, an exact McNemar $p = 0.26$ on the pooled discordant pairs, and no individual configuration reaches significance. Six tests were run without multiplicity correction, so a nominally significant single result would have warranted caution; none arose. The design bounds the effect rather than merely failing to find one. Simulating the exact test at the observed discordant rate of 1.54 percent gives 81 percent power against a net accuracy shift of one percentage point across the 1300 distinct items, and effectively complete power at 1.5 points. A directional effect of one point or larger would therefore have been detected; the evidence supports instability rather than degradation down to about that resolution.

The measurement here required a correction worth reporting, because it is a trap for anyone repeating this design. Our collection-time scorer accepted only the answer format the prompt requested, and the models frequently answered correctly in a different unambiguous form. Between 11 and 38 percent of responses were scored wrong for formatting alone, which inverted the apparent accuracy ordering across quantization levels and inflated the flip count roughly fourfold. Re-deriving every answer offline from the stored response text, with no new inference, resolves it, and comparing answers numerically rather than as strings resolves a second, smaller case. Table 3 reports the corrected figures, and the artifact ships both the original and the re-derived scores so the correction is auditable. Anyone repeating this design should validate the extractor against the model's actual output formats before drawing conclusions from flip counts.

We state the consequence plainly because it is easy to misread in either direction. Cached serving is not measurably worse at these settings, so a practitioner

choosing caching for throughput is not trading away accuracy on average. What they are trading away is repeatability: a specific item answered correctly may be answered incorrectly on a later identical request, and the reverse, with the two effects cancelling in the aggregate. Reporting only the aggregate hides this, which is the same observation made about compressed models, where matched average accuracy conceals substantial per-item disagreement [19].

Concurrent work reaches the same conclusion by a different route. Lorup [27] compares a retained live cache against a one-shot prefill of identical tokens on a reasoning benchmark, with a per-path replica control equivalent in spirit to our four-pass protocol, and likewise finds substantial suffix divergence, correctness flips in both directions, and no aggregate accuracy shift. That study adds a bidirectional cache-transplantation experiment and an FP32 falsification that we do not have; ours adds a serving-engine setting, a weight-quantization axis, and multi-turn agent episodes that it does not. We therefore present this section as independent replication rather than as a new finding, and note that two studies with different models, benchmarks, precisions, and execution stacks now agree that the outcome effect is instability without direction, in contrast to the systematic bias reported earlier by Chodavarapu and Xu [7].

V. DISCUSSION

A. WHAT PRACTITIONERS SHOULD TAKE FROM THIS

Enabling the cache changes the guarantee a serving endpoint provides. The savings are real and large [4], and nothing in our data suggests that caching makes models worse on average, so nothing here argues against prefix caching. The change to the guarantee, however, is currently undocumented and unmeasured in deployment.

Three consequences follow directly. First, a system whose behavior must be auditable or repeatable, for example one whose outputs are logged for later review, cannot obtain that property from a cache-enabled endpoint alone, because the same input is not guaranteed to reproduce the same output. Second, debugging becomes harder in a specific way: a failure observed once may not reproduce on replay, not because the input was captured incorrectly but because the server's cache state differs. Third, an A/B comparison between two prompts or two models run at different times on a shared endpoint is confounded by whatever else that endpoint served in between.

The mitigation is narrower than disabling the cache. Our isolation experiments show that reproducibility returns when cache state is controlled rather than when caching is abandoned: disabling the server-level prompt cache while leaving prefix caching on cut run-to-run divergence from 38.8 to 1.2 percent, and restarting the server between runs achieved the same on the second

engine. A workflow that needs repeatable results can therefore keep most of the throughput benefit by pinning cache configuration and resetting state at run boundaries, rather than turning the cache off entirely. A more practical middle path, which our per-request telemetry makes concrete, is to record cache exposure alongside each response so that irreproducibility is at least detectable after the fact. Only one of the two engines we tested reports this uniformly: llama.cpp exposes cached tokens on every request, while vLLM 0.11.0 leaves the field null on the endpoint we used even though the cache is demonstrably active.

B. IMPLICATIONS FOR BENCHMARK AND AGENT EVALUATION

Published evaluations of language models do not report cache configuration. Our results imply that they should. A benchmark score obtained on a cache-enabled endpoint is a sample from a distribution induced partly by serving history, not a property of the model and prompt alone, and the single-turn bridge shows that individual items change correctness at a measurable rate even when the aggregate does not move.

The effect is large enough to matter at the scale differences that evaluation papers routinely treat as meaningful. Reported variance from seeds and training noise is generally small [28], and score differences of a few points are commonly discussed as substantive. Our cross-arm measurements change agent trajectories on a majority of episodes in several configurations, and while trajectory change is not the same as score change, it is a much larger perturbation than the sources evaluation practice currently controls for.

This suggests a concrete and cheap addition to existing reproducibility checklists [29]–[31]: report the serving engine, its version, and whether prefix caching was enabled, in the same way that decoding parameters are already reported. This is the kind of configuration-level reporting that critiques of agent evaluation practice have called for [32]. Work on reliability metrics for agents measures repeated-trial variation and attributes it to the model [33]; our results indicate that part of that variation may belong to the serving layer instead.

C. RELATIONSHIP TO PRIOR SINGLE-TURN FINDINGS

The closest prior study established that cached and recomputed decoding differ numerically under FP16, using single-turn arithmetic, and reported a systematic accuracy bias across most of its conditions [7]. Our single-turn bridge reproduces the divergence half of that result and, on our models and item set, does not reproduce the directional half. Both paths are individually deterministic, they disagree on a substantial fraction of items, correctness flips occur in both directions, and aggregate accuracy does not move.

We read this as a refinement rather than a contradiction. Their setup differs from ours in models, in item set, and in the absence of a per-path determinism control, and a directional effect may well be real for their configuration. The distinction matters for what practitioners should worry about. A systematic bias would mean cached serving is quietly worse, and would call for correction. Instability without bias means cached serving is not worse on average but is not repeatable, which calls for disclosure and for care in any workflow that assumes repeatability.

VI. THREATS TO VALIDITY

Construct validity. Our primary measure is divergence in emitted token identifiers, which is a strict criterion: two runs that produce semantically identical answers in different words are counted as divergent. This is deliberate, because the question we ask is whether the system is reproducible, not whether it is approximately as good. Where the question is about quality rather than reproducibility we switch to the benchmark’s own checker and report task outcomes separately.

Internal validity. The design’s main threat is that some uncontrolled factor, rather than the cache, produces the divergence we attribute to it. One such factor was present in our first grid and we found it only by testing for it. The ten configurations of Table 2 were collected with llama.cpp’s server-level prompt cache at its default, which inflates the within-arm measurement for reasons unrelated to prefix caching; Section IV-C isolates it and reports the corrected figure. We also verified that the execution order of passes, which differed between our llama.cpp and vLLM lanes, does not affect the result. Both the original and the controlled measurements are reported rather than one silently replacing the other. The broader threat is answered by measurement rather than by argument. Every configuration is executed twice in full, and the cache-disabled repetitions are bit-identical throughout, 0 of 800 episodes, which bounds the contribution of any other nondeterminism source at 0.5 percent under our conditions. Batch size is fixed at one and requests are serial, removing batch-composition effects [11]. Cache precision is held at 16-bit floating point in all arms so that it never covaries with the weight format. Chunked prefill, where the engine enables it by default, is left at its default and is identical across arms of a configuration.

A tautology objection. A reader may respond that reusing cached state obviously changes arithmetic, so divergence is expected and the finding is definitional. We accept the mechanism as unsurprising and answer the objection with an experiment rather than an argument. Restoring the cache to a known state before each recompute pass makes both paths individually reproducible, 40 of 40 items each, while the two paths still differ from each other on 14 of 40. Divergence is therefore a function

of cache state and not of residual randomness in either path, and cache state is precisely what deployments do not control. Beyond that we claim the magnitude and the consequence. Neither the fraction of agent episodes affected, nor the interaction with quantization, nor the fact that correctness flips without a shift in aggregate accuracy, follows from knowing that floating point addition is not associative. Practice also does not treat the effect as expected: two production bug reports describe it as a defect [5], [6], and published evaluations do not report cache configuration at all.

Completeness of the released runs. Three details are visible in the artifact and we state them here. The 16-bit bridge log retains 67 records from an aborted first attempt that was restarted; the analysis keys on item and pass, so the restarted run supersedes them and the reported values are unaffected. The controlled quantization cells at 16-bit and Q3_K_M were run once rather than twice, so they contribute cross-arm values but no within-arm repeat. One vLLM cell in the ordering comparison completed without its end-of-run marker and one was not collected; the reported vLLM figures use the cells that completed, and the counts are given with each.

External validity. We study open-weight models in the 7 to 14 billion parameter range on a single consumer-class accelerator, because that is what the study's budget allowed. Behavior at frontier scale, on hosted endpoints, or under multi-tenant load with cache eviction may differ, and we make no claim about it. Our workloads are one agentic benchmark and one mathematics benchmark, both English. The direction we would expect from these limits is that multi-tenant deployments, where cache contents depend on other users' traffic, would show more history dependence rather than less, but we did not measure that and do not assert it.

Statistical conclusion validity. The agentic benchmark is difficult for models of this size, and their success rates sit near the floor. We therefore do not draw outcome conclusions from it and say so explicitly in Section IV; the trajectory-level measurements from that workload remain valid because they do not depend on task success. Outcome conclusions rest on the mathematics workload, where accuracy sits at 89 to 93 percent, well clear of the floor. Where we report a null result we report the discordant counts alongside the test, since a p-value on small counts is easy to over-read [34].

VII. CONCLUSION

Prefix caching is enabled by default in the serving stacks that most deployments use, and it is treated as a transparent optimization. This paper shows that it is not transparent. Holding decoding, seed, batch size, and request order fixed, we found that repeated execution of an identical workload was bit-identical when the cache is disabled, in all ten configurations across two

independent engines, 0 of 800 episodes, and was not when the cache is enabled. Three further experiments locate the dependence precisely. It is not execution order. It is state carried between runs, and on one engine a single prompt-cache flag moves run-to-run divergence by 37.5 percentage points. When cache state is restored to a known point, both the cached and the recompute paths reproduce exactly, while continuing to differ from each other. Cached serving is deterministic given cache state; the problem is that cache state is invisible to the request, absent from the response, and uncontrolled by default.

The magnitude depends on how the model is quantized and on how long the episode is. Coarser weight quantization widens the gap between cached and recomputed execution, from 36.2 percent of agent episodes at 16-bit to 75.0 percent at four-bit, and single-turn requests that are nearly unaffected at full precision diverge on more than forty percent of items once quantized. Longer episodes amplify it again, since a single changed token propagates through every subsequent turn.

What the effect does to task outcomes is narrower than one might assume. Individual answers change correctness in both directions and aggregate accuracy does not move. Cached serving is not worse. It is not repeatable. Those are different problems requiring different responses. The first would call for a correction; the second calls for disclosure.

The immediate recommendation is cheap. Evaluations and deployment records should state the serving engine, its version, and whether prefix caching was enabled, alongside the decoding parameters that are already reported as a matter of course. Engines should report cache exposure per request uniformly, which one of the two we tested does and the other does not, so that irreproducibility is at least detectable after the fact. Longer term, deterministic modes that cover cached execution already exist in at least one stack [12], and the underlying mechanism is understood [11], [35]. What our measurements add is the size of the gap those modes are closing, in the default configuration that most deployments and nearly all published evaluations actually run [36], [37].

DATA AVAILABILITY

The measurement harness, the complete raw per-request logs, and the analysis pipeline are available at <https://github.com/aditi-p31/cache-divergence-study>. The logs record, for every request, the prompt hash, the emitted token identifiers with log probabilities, the latency, and the cache exposure reported by the serving engine. Every number and figure in this paper is regenerated from those logs by the scripts in `analysis/`: `reextract_bridge.py` derives the single-turn answers, `analyze.py` and `analyze_repair.py` write findings.json and `repair_findings.json`, and

make_tables.py and make_figures.py produce the tables and figures from those two files. Pinned engine versions, model checksums, and the launch flags used for each configuration are recorded in the repository, and are load-bearing rather than incidental for this particular study.

ACKNOWLEDGMENT

The author used Claude (Anthropic) to assist with measurement and analysis code, figure generation, and manuscript preparation. The author designed the study, verified all reported results against the raw data and primary sources, and takes full responsibility for the content. All hypotheses, interpretations, and conclusions presented in this study reflect the author's original ideas.

REFERENCES

- [1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), 2023, arXiv:2309.06180.
- [2] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in Advances in Neural Information Processing Systems 37 (NeurIPS), 2024. [Online]. Available: <https://arxiv.org/abs/2312.07104>
- [3] I. Gim, G. Chen, S.-s. Lee, N. Sarda, A. Khandelwal, and L. Zhong, "Prompt cache: Modular attention reuse for low-latency inference," in Proceedings of Machine Learning and Systems (MLSys), 2024, arXiv:2311.04934.
- [4] E. Lumer, F. Nizar, A. Jangiti, K. Frank, A. Gulati, M. Phadate, and V. K. Subbiah, "Don't break the cache: An evaluation of prompt caching for long-horizon agentic tasks," 2026. [Online]. Available: <https://arxiv.org/abs/2601.06007>
- [5] vLLM Project Contributors, "Accuracy degradation in vLLM when prefix-cache is enabled for recomputation workloads," GitHub issue #18055, vllm-project/vllm, May 2025, accessed: 2026-08-16. [Online]. Available: <https://github.com/vllm-project/vllm/issues/18055>
- [6] —, "Prefix caching produces different output on first request (cache miss) vs subsequent requests (cache hit)," GitHub issue #33123, vllm-project/vllm, Jan. 2026, accessed: 2026-08-16. [Online]. Available: <https://github.com/vllm-project/vllm/issues/33123>
- [7] R. Chodavarapu and L. Xu, "The illusion of equivalence: Systematic FP16 divergence in KV-cached autoregressive inference," 2026. [Online]. Available: <https://arxiv.org/abs/2604.15409>
- [8] D. Pape, J. Evertz, and L. Schönherr, "The silent hyperparameter: Quantifying the impact of inference backends on LLM reproducibility," 2026. [Online]. Available: <https://arxiv.org/abs/2605.19537>
- [9] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez, "The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models," in Proceedings of the 42nd International Conference on Machine Learning (ICML), ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 48 371–48 392. [Online]. Available: <https://proceedings.mlr.press/v267/patil25a.html>
- [10] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," 2021. [Online]. Available: <https://arxiv.org/abs/2110.14168>
- [11] H. He and Thinking Machines Lab, "Defeating nondeterminism in LLM inference," Thinking Machines Lab blog, Sep. 2025, accessed: 2026-08-16. [Online]. Available: <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>
- [12] SGLang Team, "Towards deterministic inference in SGLang and reproducible RL training," LMSYS Org blog, Sep. 2025, accessed: 2026-08-16. [Online]. Available: <https://lmsys.org/blog/2025-09-22-sglang-deterministic/>
- [13] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, "CacheBlend: Fast large language model serving for RAG with cached knowledge fusion," in Proceedings of the Twentieth European Conference on Computer Systems (EuroSys), 2025.
- [14] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, "H2O: Heavy-hitter oracle for efficient generative inference of large language models," in Advances in Neural Information Processing Systems 36 (NeurIPS), 2023, arXiv:2306.14048.
- [15] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in International Conference on Learning Representations (ICLR), 2024, arXiv:2309.17453.
- [16] B. Atil, S. Aykent, A. Chittams, L. Fu, R. J. Passonneau, E. Radcliffe, G. R. Rajagopal, A. Sloan, T. Tudrej, F. Ture, Z. Wu, L. Xu, and B. Baldwin, "Non-determinism of "deterministic" LLM settings," 2024. [Online]. Available: <https://arxiv.org/abs/2408.04667>
- [17] R. Gond, A. K. Kamath, R. Ramjee, and A. Panwar, "LLM-42: Enabling determinism in LLM inference with verified speculation," 2026. [Online]. Available: <https://arxiv.org/abs/2601.17768>
- [18] E. Kurtic, A. N. Marques, S. Pandit, M. Kurtz, and D. Alistarh, "Give me BF16 or give me death? accuracy-performance trade-offs in LLM quantization," in Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), 2025, arXiv:2411.02355.
- [19] A. Dutta, S. Krishnan, N. Kwatra, and R. Ramjee, "Accuracy is not all you need," in Advances in Neural Information Processing Systems 37 (NeurIPS), 2024, arXiv:2407.09141.
- [20] K. Marchisio, S. Dash, H. Chen, D. Aumiller, A. Üstün, S. Hooker, and S. Ruder, "How does quantization affect multilingual LLMs?" in Findings of the Association for Computational Linguistics: EMNLP, 2024, arXiv:2407.03211.
- [21] G. Gerganov and llama.cpp contributors, "llama.cpp," GitHub repository, ggml-org/llama.cpp, MIT License, 2023, accessed: 2026-08-16. [Online]. Available: <https://github.com/ggml-org/llama.cpp>
- [22] G. Gerganov and ggml contributors, "GGUF file format specification," ggml-org/ggml, docs/gguf.md, 2023, accessed: 2026-08-16. [Online]. Available: <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>
- [23] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," Psychometrika, vol. 12, no. 2, pp. 153–157, 1947.
- [24] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," Neural Computation, vol. 10, no. 7, pp. 1895–1923, 1998.
- [25] G. Gerganov and llama.cpp contributors, "server: host-memory prompt caching," Pull request #16391, ggml-org/llama.cpp, merged 2025-10-09, 2025, accessed: 2026-08-18. [Online]. Available: <https://github.com/ggml-org/llama.cpp/pull/16391>
- [26] llama.cpp contributors, "llama.cpp server documentation," tools/server/README.md, ggml-org/llama.cpp, 2026, accessed: 2026-08-18. [Online]. Available: <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md>
- [27] A. B. Lorup, "Stage-replay divergence follows the KV cache: Fixed-prefix precision controls and bidirectional cache transplantation," 2026, accessed: 2026-08-16. [Online]. Available: <https://arxiv.org/abs/2607.28495>
- [28] L. Madaan, A. K. Singh, R. Schaeffer, A. Poulton, S. Koyejo, P. Stenetorp, S. Narang, and D. Hupkes,

- “Quantifying variance in evaluation benchmarks,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.10229>
- [29] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d’Alché Buc, E. Fox, and H. Larochelle, “Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program),” *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021. [Online]. Available: <https://jmlr.org/papers/v22/20-303.html>
- [30] S. Biderman, H. Schoelkopf, L. Sutawika, L. Gao, J. Tow, B. Abbasi, A. F. Aji, P. S. Ammanamanchi, S. Black, J. Clive, A. DiPofi, J. Etxaniz, B. Fattori, J. Z. Forde, C. Foster, J. Hsu, M. Jaiswal, W. Y. Lee, H. Li, C. Lovering, N. Muennighoff, E. Pavlick, J. Phang, A. Skowron, S. Tan, X. Tang, K. A. Wang, G. I. Winata, F. Yvon, and A. Zou, “Lessons from the trenches on reproducible evaluation of language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.14782>
- [31] Y. Zhu, T. Jin, Y. Pruksachatkun, A. Zhang, S. Liu, S. Cui, S. Kapoor, S. Longpre, K. Meng, R. Weiss, F. Barez, R. Gupta, J. Dhamala, J. Merizian, M. Giulianelli, H. Coppock, C. Ududec, J. Sekhon, J. Steinhardt, A. Kellermann, S. Schwettmann, M. Zaharia, I. Stoica, P. Liang, and D. Kang, “Establishing best practices for building rigorous agentic benchmarks,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.02825>
- [32] S. Kapoor, B. Stroebel, Z. S. Siegel, N. Nadgir, and A. Narayanan, “AI agents that matter,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.01502>
- [33] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “ τ -bench: A benchmark for tool-agent-user interaction in real-world domains,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.12045>
- [34] E. Miller, “Adding error bars to evals: A statistical approach to language model evaluations,” 2024. [Online]. Available: <https://arxiv.org/abs/2411.00640>
- [35] J. Yuan, H. Li, X. Ding, W. Xie, Y.-J. Li, W. Zhao, K. Wan, J. Shi, X. Hu, and Z. Liu, “Understanding and mitigating numerical sources of nondeterminism in LLM inference,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.09501>
- [36] Y. Yao, X. Tan, C.-H. Liu, Y. Li, Z. Wang, W. Yu, Z. Tan, Y. Tian, G. Zhao, L. Sun, X. Zhang, and T. Yang, “Harness-bench: Measuring harness effects across models in realistic agent workflows,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.27922>
- [37] K. Chu, Y. Zhou, and W. Zhang, “Margin-Gate: Sparse margin-triggered verification for batch-invariant LLM inference,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.30218>



ADITI PATODIYA (Senior Member, IEEE) received the B.Tech. degree in Computer Engineering from Charotar University of Science and Technology, India, and the M.S. degree in Computer Science from California State University, Long Beach, CA, USA. She is a Senior Software Engineer based in California with over 10 years of industry experience, a reviewer for computing venues including CHI, Supercomputing, ECIS, and

ISMAR, and a technical writer. Her expertise spans enterprise AI infrastructure, context engineering, and software engineering for large language model systems. Her work focuses on designing and architecting scalable backend infrastructure and context pipelines capable of handling large-scale production traffic. Her current research centers on LLM inference reproducibility, prompt caching behaviors, and the measurement of agentic systems in production deployments.